PRACTICAL 5

```
nishanth@LAPTOP-6Q3ONO7H:~$ cd OSLAB
nishanth@LAPTOP-6Q3ONO7H:~/OSLAB$ mkdir pipes
nishanth@LAPTOP-6Q3ONO7H:~/OSLAB$ cd pipes
nishanth@LAPTOP-6Q3ONO7H:~/OSLAB/pipes$ nano producer.c
nishanth@LAPTOP-6Q3ONO7H:~/OSLAB/pipes$ gcc producer.c -o producer
nishanth@LAPTOP-6Q3ONO7H:~/OSLAB/pipes$ ./producer
Parent producing data: Hello from Parent Process
Child consumed data: Hello from Parent Process
nishanth@LAPTOP-6Q3ONO7H:~/OSLAB/pipes$ nano pipegrep.c
nishanth@LAPTOP-6Q3ONO7H:~/OSLAB/pipes$ gcc pipegrep.c -o pipegrep
nishanth@LAPTOP-6Q3ONO7H:~/OSLAB/pipes$ ./pipegrep
-rw-r--r-- 1 nishanth nishanth  1184 Aug 22 04:04 pipegrep.c
-rwxr-xr-x 1 nishanth nishanth 16344 Aug 22 04:03 producer
-rw-r--r-- 1 nishanth nishanth   874 Aug 22 04:03 producer.c
```

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid;
    char message[] = "Hello from Parent Process";
    char buffer[100];

    // Create pipe
    if (pipe(fd) == -1)
    {
        perror("Pipe failed");
        return 1;
    }

    // Create child process
    pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if (pid > 0)
    {
        // Parent = Producer
        close(fd[0]);

        printf("Parent producing data: %s\n", message);

        write(fd[1], message, strlen(message) + 1);

        close(fd[1]);

        wait(NULL);
    }
    else
    {
        // Child = Consumer
        close(fd[1]);

        read(fd[0], buffer, sizeof(buffer));

        printf("Child consumed data: %s\n", buffer);
```

```
        close(fd[0]);
    }

    return 0;
}
```

```
nishanth@LAPTOP-6Q3ONO7H:~/OSLAB/pipes$ cat producer.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid;
    char message[] = "Hello from Parent Process";
    char buffer[100];

    // Create pipe
    if (pipe(fd) == -1)
    {
        perror("Pipe failed");
        return 1;
    }

    // Create child process
    pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if (pid > 0)
    {
        // Parent = Producer
        close(fd[0]);

        printf("Parent producing data: %s\n", message);

        write(fd[1], message, strlen(message) + 1);

        close(fd[1]);

        wait(NULL);
    }
    else
    {
        // Child = Consumer
        close(fd[1]);

        read(fd[0], buffer, sizeof(buffer));
```

```
        printf("Child consumed data: %s\n", buffer);

        close(fd[0]);
    }

    return 0;
}
```